

AI-Driven Multi-Chain DEX Trading & Atomic Arbitrage System

A technical write-up of the most sophisticated DeFi automation I have designed and built: an AI-agent-driven, multi-step, on-chain transaction pipeline spanning six EVM networks, with an LLM decision layer, MEV-protected execution, and an on-chain profit guarantee.

Repository: github.com/AlIMertBOSTAN/ai-trade-bot

Honesty note up front: every number in Sec. 4 comes from backtests on real exchange data and paper-trading runs of the full pipeline. The live-execution path (signed transactions, Flashbots relay, the deployed-contract flow) is fully implemented and verified by compilation, static analysis and pre-flight simulation, but is deliberately gated behind fail-safe checks — this document claims engineering outcomes, not realized trading profits.

1. Objectives

I built this system to solve three concrete problems with one autonomous agent:

Cross-chain DEX arbitrage. The same asset (e.g. WETH) trades at different prices on different EVM networks and venues — Uniswap v2/v3 on Ethereum, Arbitrum, Base and Optimism, PancakeSwap on BNB Chain, QuickSwap on Polygon. The objective was to continuously scan all venues in parallel and surface only opportunities that remain **net profitable after gas, slippage and bridging costs**, then execute the two legs as a single atomic on-chain transaction. -> engine/arbitrage/scanner.py, contracts/ArbExecutor.sol

Hybrid AI directional signals. Pure rule-based indicators (RSI, EMA crossover, MACD, momentum) are noisy; a pure LLM is inconsistent and unauditable. The objective was a **hybrid fusion**: the technical layer produces a candidate decision and confidence, the LLM validates or vetoes it with a structured JSON verdict, and the two are combined with explicit agreement/disagreement rules. The LLM also acts as a **market analyst agent**: it ingests live CEX vs DEX price comparisons (Binance public REST vs on-chain pools) plus real-time news headlines from RSS feeds and produces a structured sentiment/risk commentary. -> engine/signals/engine.py, engine/signals/llm.py, engine/marketdata/analyst.py

Removing the human from the loop — except where the human must stay. Price polling across 6 chains, indicator computation, signal generation, risk gating, order execution, mark-to-market and persistence all run in one supervised loop. The human sets parameters and chooses the mode (paper <-> live); everything else is autonomous. The design invariant throughout: **capital preservation precedes returns**. -> engine/bot/orchestrator.py

2. Code Stack

The architecture is deliberately polyglot — each language where it is strongest, with a hard boundary between them (the Python engine exposes REST + WebSocket on 127.0.0.1:8787; the TypeScript side only consumes that API):

Layer	Technology	Why
Chain reads, pricing, order execution	Python 3.10+, web3.py	Mature web3 ecosystem; parallel RPC via thread pool
AI / LLM advisor & market analyst	Python + Anthropic / OpenAI SDKs	Provider-agnostic complete() layer; signal logic co-located with data
Open market data	Binance public REST + DexScreener (stdlib urllib, TTL-cached)	Key-less CEX/DEX price comparison, order-book imbalance, pool liquidity
Real-time news	RSS (CoinDesk, Cointelegraph, Decrypt, The Defiant) , stdlib XML	Headline context for the LLM analyst; per-feed fail-safe
Backend API	FastAPI + Uvicorn (WebSocket)	Low-latency REST + live event stream to the UI
Desktop UI	Electron + TypeScript + React + Chart.js	Type-safe dashboard: equity curve, positions, arbitrage feed, CEX/DEX spreads, news

Layer	Technology	Why
Keeper / MEV-protected submission	TypeScript + Ethers.js v6	Flashbots Protect private relay; staticCall pre-flight simulation
Atomic arbitrage contract	Solidity ^0.8.24	Two-leg swap in one transaction, on-chain revert-on-no-profit
Persistence	SQLite	Trades, signals, equity history, backtests

Scope: ~57 source files, ~3,400 lines across Python, TypeScript and Solidity.

3. Risk Controls

Risk management is the core of the system — a layered defense where every rejection carries an explicit reason (no silent failures):

- **Slippage ceiling.** Every swap carries `amountOutMinimum` derived from a configurable bps tolerance; the router reverts if exceeded. -> `RiskManager.min_out()`, `ArbExecutor.sol`
- **Gas price ceiling.** Spot gas is read before execution; above `max_gas_gwei` the transaction is **never sent**. -> `RiskManager.gas_ok()`, `flashbotsKeeper.ts`
- **Atomicity + revert-on-no-profit.** Both arbitrage legs execute inside one transaction. If the realized profit at the end of the call is below `minProfit`, the contract reverts the **entire transaction** with `Unprofitable()` — no half-filled positions, profit enforced on-chain rather than trusted off-chain. -> `ArbExecutor.sol::executeArb()`
- **MEV / sandwich protection.** Live transactions bypass the public mempool via **Flashbots Protect**, with an `staticCall` pre-flight so reverting transactions never reach the chain. -> `src/core/keeper/flashbotsKeeper.ts`
- **Daily-loss kill-switch.** Realized intraday loss beyond `max_daily_loss_usd` halts all new trading. -> `RiskManager.kill_switch_triggered()`
- **Position limits + stop-loss / take-profit.** Max notional per position, max concurrent positions, automatic SL/TP triggers on every tick. -> `RiskManager.evaluate()`, `check_stop_take()`
- **Confidence gating.** Signals below `min_confidence` are rejected before they ever reach sizing.
- **Fail-safe mode switching + deadlines.** Live mode is unreachable without a wallet key (`assert_live_ready()`); every router call carries a deadline so stale transactions cannot fill at bad prices. The LLM layer is also fail-safe: if the provider is down or unconfigured, the engine falls back to the pure technical decision rather than halting.

4. Outcomes

- **Backtest on real exchange data** (Binance ETHUSDT 1h klines, 300 candles, run June 2026 — a -19.4% bear segment): the strategy returned **-0.34% vs -19.43% buy-and-hold (+19.1pp outperformance)** with a **maximum drawdown of 0.85%** across 5 trades. The risk gates kept the bot mostly flat in a falling market — exactly the designed behavior.
- **Synthetic-series backtest** (300 candles, trending regime): **+3.3% total return, 2.9% max drawdown, Sharpe 1.36, 67% win rate** over 21 trades, with SL/TP triggering automatically.
- **Arbitrage detection verified end-to-end:** in a controlled test the scanner found a WETH buy @ \$2,995 (Polygon/QuickSwap) -> sell @ \$3,030 (Arbitrum/Uniswap) opportunity, a **1.17% spread netting \$13.45 after estimated costs**, while correctly discarding below-threshold spreads.
- **Live data-plane verified against production APIs:** real-time CEX/DEX comparison returns coherent spreads (ETH -10.8 bps, BTC -40.6 bps, ARB -27.4 bps between Binance and the most liquid Uniswap pools), and the news pipeline aggregates three independent feeds with per-feed fault isolation.
- **Risk gates exercised:** low-confidence signals, position-limit breaches and the kill-switch scenario were all rejected with explicit reasons.
- **Engineering quality:** all Python modules compile and import cleanly; TypeScript passes `--strict` typechecking and a production `electron-vite` build; the Solidity contract compiles under `solc 0.8.26` with **zero warnings**.
- **Operational visibility:** a real-time dashboard streams equity, open positions, multi-chain prices, arbitrage opportunities, hybrid signals, CEX/DEX spreads and live news over WebSocket.

5. Representative Code

Hybrid signal fusion — the heart of the AI agent: technical pre-filter, LLM verdict, explicit agreement/conflict resolution, fail-safe fallback (engine/signals/engine.py):

```
def generate_signal(chain_id, base, quote, closes):
    # 1) Rule-based candidate from technical indicators (pre-filter)
    tech = compute_snapshot(closes)          # RSI, EMA, MACD, momentum
    rule_action, rule_conf = _rule_decision(tech) # -> ("BUY"/"SELL"/"HOLD", conf)

    # 2) LLM advisor validates / overrides with a structured JSON verdict
    advice = llm.advise(base, quote, tech, rule_action, _returns(closes))

    if advice and advice["action"] in ("BUY", "SELL", "HOLD"):
        llm_action, llm_conf = advice["action"], float(advice["confidence"])
        if llm_action == rule_action:
            # Both layers AGREE -> boost confidence
            action, confidence = rule_action, min(1.0, 0.5*rule_conf + 0.5*llm_conf + 0.1)
        else:
            # CONFLICT -> defer to the more cautious LLM, cut confidence
            action, confidence = llm_action, max(0.0, 0.5 * llm_conf)
        source = "hybrid"
    else:
        # LLM unavailable/failed -> pure technical decision (fail-safe)
        action, confidence, source = rule_action, rule_conf, "technical"

    return TradeSignal(chain_id, base, quote, action, confidence,
                       tech, advice and advice.get("rationale", ""), source)
```

The risk gate every signal must pass before becoming an order (engine/risk/manager.py):

```
def evaluate(self, signal, open_positions, cash_usd):
    if self.kill_switch_triggered():          # daily-loss kill-switch
        return RiskDecision(False, "daily loss limit hit")
    if signal.confidence < self.risk.min_confidence: # confidence gate
        return RiskDecision(False, "confidence below threshold")
    if signal.action == "BUY":
        if len(open_positions) >= self.risk.max_open_positions:
            return RiskDecision(False, "max positions reached")
        size = min(self.risk.max_position_usd, cash_usd * 0.95) # notional cap
        return RiskDecision(True, "approved", size_usd=size)
    ...
```

On-chain profit enforcement — the contract refuses to lose money; if the two-leg arbitrage ends below the threshold, the whole transaction is rolled back (contracts/ArbExecutor.sol):

```
// Both swap legs executed atomically above; now enforce profit on-chain:
uint256 profit = endBal > startBal ? endBal - startBal : 0;
if (profit < minProfit) revert Unprofitable(profit, minProfit);
emit ArbExecuted(token, amountIn, endBal, profit);
```

MEV-protected submission with pre-flight simulation (src/core/keeper/flashbotsKeeper.ts):

```
// 1) Pre-flight: simulate the exact call; if it would revert, never send it
await arbExecutor.executeArb.staticCall(params)

// 2) Submit via Flashbots Protect (private relay) – invisible to
//     front-running / sandwich bots in the public mempool
const provider = new JsonRpcProvider('https://rpc.flashbots.net')
const tx = await wallet.connect(provider).sendTransaction(signedArbTx)
```

Full source: github.com/AlimertBOSTAN/ai-trade-bot — engine/ (Python core), src/ (TypeScript UI + keeper), contracts/ (Solidity). A reproducible real-data backtest is available via `python -m engine.backtest.run_live_backtest --symbol ETHUSDT --interval 1h --limit 300`.